

Linear Regression Networks

Machine Learning: From Scratch to Mastery

Yasser Hifny

Faculty of Computers and Artificial Intelligence
Capital University

Outline

- Linear regression as a fundamental supervised learning model.
- Linear regression networks and their matrix representation.
- The learning problem and Mean Square Error (MSE) loss.
- Analytical solution using the normal equation.
- Gradient-based numerical solution.
- Mini-batch stochastic gradient descent.
- A two-output, three-input worked example.
- Python implementation and assignment.

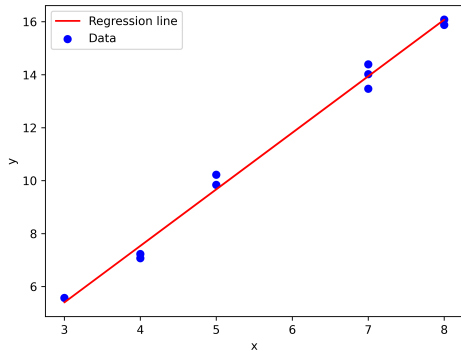
The Linear Regression Model

- Linear regression models the relationship between input variables and output variables using a linear function.
- Inputs may be float, binary, or integer values.
- Outputs must be real-valued (float) variables.
- Example: predicting house price from features such as number of rooms, area, and number of bathrooms.

Key idea

The model learns a linear mapping from the input feature vector to the output vector.

One-Dimensional Linear Regression



Linear function between one input variable and one output variable fitted using linear regression.

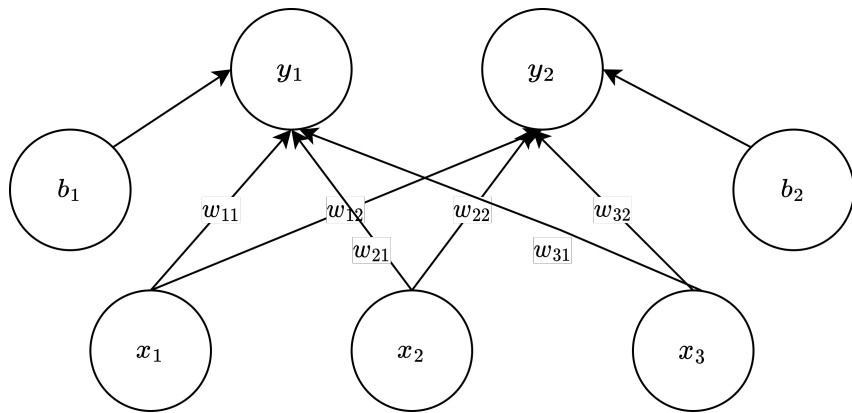
A Linear Regression Network

- Consider three input variables and two output variables.
- Every output is connected to every input.

$$y_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1,$$

$$y_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2.$$

Linear Regression Network Architecture



Linear regression network with three input nodes and two output nodes.

Linear regression network with three

Matrix Representation

The two-output model can be written as

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}.$$

Generally,

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

where $\mathbf{W} \in \mathbb{R}^{K \times d}$ and $\mathbf{b} \in \mathbb{R}^K$ are learned parameters.

The Learning Problem

Given training data

$$\{(\mathbf{x}_1, \mathbf{t}_1), \dots, (\mathbf{x}_N, \mathbf{t}_N)\},$$

the goal is to estimate $\mathbf{W}$ and $\mathbf{b}$ so that predictions $\mathbf{y}$ are close to targets $\mathbf{t}$. For sample n ,

$$E^n(\mathbf{W}, \mathbf{b}) = \frac{1}{2} \sum_{k=1}^K (y_k^n - t_k^n)^2.$$

The total objective is the mean over all training samples.

Mean Square Error Loss

$$E(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N E^n$$

- E is the Mean Square Error (MSE) loss function.
- It is appropriate because the outputs of linear regression are real-valued.
- Training seeks parameter values that minimize the loss.

Learning objective

Find the values of $\mathbf{W}$ and $\mathbf{b}$ that give the smallest training MSE.

Analytical Solution: Matrix Form

For a multiple-regression problem with one output, augment the input matrix with a bias feature:

$$E(\boldsymbol{\theta}) = \frac{1}{2N}(\mathbf{X}\boldsymbol{\theta} - \mathbf{t})^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{t}).$$

Ignoring the constant factor and expanding,

$$E(\boldsymbol{\theta}) = \boldsymbol{\theta}^T \mathbf{X}^T \mathbf{X} \boldsymbol{\theta} - 2\mathbf{t}^T \mathbf{X} \boldsymbol{\theta} + \mathbf{t}^T \mathbf{t}.$$

Normal Equation

Differentiate the quadratic loss with respect to θ :

$$\frac{\partial E}{\partial \theta} = 2X^T X \theta - 2X^T t = 0.$$

Therefore,

$$X^T X \theta = X^T t$$

and the analytical solution is

$$\theta = (X^T X)^{-1} X^T t.$$

Why Use a Numerical Solution?

- The analytical solution provides the optimum for the quadratic MSE objective when the inverse exists.
- Computing the solution directly becomes less attractive as the amount of training data and dimensionality grow.
- We therefore focus on gradient descent as a numerical solution.
- Mini-batch stochastic gradient descent makes the method suitable for larger datasets.

Gradient of the Weight Parameters

For weight w_{rs} ,

$$\frac{\partial E^n}{\partial w_{rs}} = \frac{\partial E^n}{\partial y_r} \frac{\partial y_r}{\partial w_{rs}} = (y_r^n - t_r^n) x_s^n.$$

Averaging over the training set gives

$$\boxed{\frac{\partial E}{\partial w_{rs}} = \frac{1}{N} \sum_{n=1}^N (y_r^n - t_r^n) x_s^n.}$$

Gradient of the Bias Parameters

For bias b_r ,

$$\frac{\partial E}{\partial b_r} = \frac{1}{N} \sum_{n=1}^N (y_r^n - t_r^n).$$

The gradients quantify how the MSE changes when each parameter is changed.

Gradient Descent Updates

The parameters are updated according to

$$w_{rs}^{t+1} = w_{rs}^t - \eta \frac{\partial E}{\partial w_{rs}},$$
$$b_r^{t+1} = b_r^t - \eta \frac{\partial E}{\partial b_r}.$$

where η is the learning rate.

Interpretation

Each update moves the parameters in the direction opposite to the loss gradient.

Mini-Batch Stochastic Gradient Descent

- 1 Initialize $\mathbf{W}$ and $\mathbf{b}$ randomly or with zeros.
- 2 Divide the training data into mini-batches.
- 3 For each batch, compute predictions.
- 4 Compute the batch MSE.
- 5 Compute gradients with respect to weights and biases.
- 6 Update the parameters using gradient descent.
- 7 Repeat until convergence or a maximum number of epochs.

Advantages and Hyperparameters

- Can handle datasets that do not fit entirely in memory.
- Can exploit parallelism and vectorization.
- Provides a practical numerical approach to the quadratic optimization problem.

Trade-off

Mini-batch gradient descent requires tuning the batch size and learning rate.

Worked Example: Training Data

We use 10 synthesized training samples with three input features and two output targets.

x_1	x_2	x_3	t_1	t_2
0.23	0.87	0.95	2.74	1.82
0.34	0.56	0.29	1.42	2.61
0.98	0.68	0.05	3.12	2.49
0.51	0.24	0.64	1.63	1.95
0.42	0.75	0.49	2.84	2.37
0.89	0.11	0.93	2.17	1.68
0.14	0.91	0.25	1.96	2.55
0.76	0.41	0.53	2.29	2.24
0.66	0.04	0.08	1.27	1.35
0.45	0.82	0.76	2.86	2.74

Python Implementation: Analytical Solution

```
def analytical_solution(X, T):  
    X_b = np.c_[np.ones((X.shape[0], 1)), X]  
    theta = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(T)  
    return theta  
  
theta_analytical = analytical_solution(X, T)  
print("Analytical solution coefficients:", theta_analytical)
```

Python Implementation: Gradient Descent

```
def gradient_descent(X, T, eta=0.1, iterations=100000):  
    m, n = X.shape  
    theta = np.random.randn(n + 1, T.shape[1])  
    X_b = np.c_[np.ones((m, 1)), X]  
    for i in range(iterations):  
        gradients = 2 / m * X_b.T.dot(X_b.dot(theta) - T)  
        theta -= eta * gradients  
    return theta  
  
theta_gradient_descent = gradient_descent(X, T)
```

Python Implementation: Comparing Results

```
Y_analytical = np.c_[np.ones((X.shape[0], 1)), X].dot(theta_analytical)
Y_gradient_descent = np.c_[np.ones((X.shape[0], 1)), X].dot(theta_gradient_descent)

mse_analytical = mean_squared_error(T, Y_analytical)
mse_gradient_descent = mean_squared_error(T, Y_gradient_descent)

print("Mean Squared Error - Analytical Solution:", mse_analytical)
print("Mean Squared Error - Gradient Descent:", mse_gradient_descent)
```

Example: Main Result

A note on the results

Since linear regression has a unique solution, both the analytical and gradient descent methods have the same result.

- The analytical approach directly computes the optimum.
- Gradient descent approaches the same optimum numerically.
- The choice of method depends on computational scale and implementation requirements.

Summary

- Linear regression is a single-layer fully connected linear network.
- The model is $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$.
- MSE is used as the objective for real-valued outputs.
- The quadratic MSE objective admits an analytical solution.
- Gradient descent provides a numerical optimization method.
- Mini-batch stochastic gradient descent reduces memory and computational requirements for large datasets.
- The example demonstrates agreement between analytical and gradient-descent solutions.

Assignment

House Price Prediction

Predict house price using Python from scratch, using the Keras deep learning library, and Google Colab.

Use the dataset-loading method specified in the chapter:

```
tf.keras.datasets.boston_housing.load_data
```

Thank You

Questions?